Below is an **end-to-end lab manual** to do this on **Minikube**:

Goal:

- Create a **user** (client cert based)
- Create a **namespace-scoped RBAC** permission to Pods with **only**: get, list, watch
- **Test** access (allowed + denied cases)

Lab Prereqs

- Minikube running and kubectl working

minikube status

kubectl version --client

kubectl cluster-info

- You should be using the **minikube context**

kubectl config current-context

expected: minikube

---

1) Create a namespace + sample pods

1.1 Create namespace

kubectl create namespace dev

1.2 Deploy a sample app (pods will be created)

kubectl -n dev create deployment nginx --image=nginx --replicas=2

kubectl -n dev get pods -o wide

Also create something in another namespace to test isolation:

kubectl -n default create deployment web --image=nginx --replicas=1

kubectl -n default get pods

---

2) Create a Kubernetes "user" using client certificates (recommended for Minikube labs)

Kubernetes does not store “users” as objects like ServiceAccounts.
For a real “user”, we typically create a **client certificate** and put it into kubeconfig.

We’ll create:

- user name: dev-user
- namespace to access: dev

2.1 Set variables

USER=dev-user

NAMESPACE=dev

CLUSTER=minikube

2.2 Generate private key + CSR

openssl genrsa -out ${USER}.key 2048

openssl req -new -key ${USER}.key -out ${USER}.csr -subj "/CN=${USER}/O=dev-team"

2.3 Find Minikube cluster CA cert + key path

Minikube typically stores CA here:

- ~/.minikube/ca.crt
- ~/.minikube/ca.key

Verify:

ls -l ~/.minikube/ca.crt ~/.minikube/ca.key

2.4 Sign the CSR with Minikube CA

```
openssl x509 -req -in ${USER}.csr \
  -CA ~/.minikube/ca.crt -CAkey ~/.minikube/ca.key -CAcreateserial \
  -out ${USER}.crt -days 365
```

Confirm cert subject:

openssl x509 -in ${USER}.crt -noout -subject

should show CN=dev-user

---

3) Add this user into kubeconfig (new context)

3.1 Create user entry (client cert auth)

kubectl config set-credentials ${USER} --client-certificate=$(pwd)/${USER}.crt --client-key=$(pwd)/${USER}.key --embed-certs=true

3.2 Create a context scoped to namespace dev

```
kubectl config set-context ${USER}-${NAMESPACE} \
  --cluster=${CLUSTER} \
  --user=${USER} \
  --namespace=${NAMESPACE}
```

3.3 Verify contexts

kubectl config get-contexts

---

4) Create RBAC: Role + RoleBinding (pods: get, list, watch only)

4.1 Create Role YAML

Create a file named: pod-reader-role.yaml

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: dev
rules:
  - apiGroups: [""]
    resources: ["pods"]
    verbs: ["get", "list", "watch"]
```

Apply:

kubectl apply -f pod-reader-role.yaml

4.2 Create RoleBinding YAML (bind role to the user identity)

Create file: pod-reader-binding.yaml

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: pod-reader-binding
  namespace: dev
subjects:
  - kind: User
    name: dev-user
    apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

Apply:

kubectl apply -f pod-reader-binding.yaml

---

5) Test access (ALLOW + DENY)

5.1 Switch to the new user context

kubectl config use-context ${USER}-${NAMESPACE}

kubectl config current-context

5.2 Positive tests (should work in dev)

kubectl get pods

kubectl get pods -o wide

kubectl get pod -l app=nginx

Pick one pod name and test get:

POD=$(kubectl get pods -o jsonpath='{.items[0].metadata.name}')

kubectl get pod ${POD} -o yaml

Watch pods (Ctrl+C to stop):

kubectl get pods -w

5.3 RBAC verification using auth can-i (very important for labs)

kubectl auth can-i list pods

kubectl auth can-i get pods

kubectl auth can-i watch pods

Expected: yes for all three.

---

6) Negative tests (must be denied)

6.1 Try actions not granted (should FAIL)

Create pod

kubectl run test --image=nginx

Delete

kubectl delete pod ${POD}

Exec

kubectl exec -it ${POD} -- sh

Expected: Error from server (Forbidden): ...

6.2 Try accessing other namespaces (should FAIL)

Try default namespace pods (even list should fail):

kubectl get pods -n default

Expected: Forbidden.

Also validate with can-i:

kubectl auth can-i list pods -n default

Expected: no

---

7) Switch back to admin

kubectl config use-context minikube

---

8) Cleanup (optional)

kubectl delete namespace dev

kubectl delete deployment web -n default

kubectl config delete-context ${USER}-${NAMESPACE}

kubectl config delete-user ${USER}

rm -f ${USER}.key ${USER}.csr ${USER}.crt ${USER}.srl

rm -f pod-reader-role.yaml pod-reader-binding.yaml

Quick Troubleshooting

A) "Forbidden" even for dev pods

Common cause: **RoleBinding subject name doesn't match cert CN**.

Check what your certificate CN is:

openssl x509 -in dev-user.crt -noout -subject

Your RoleBinding must use:

subjects:

- kind: User

  name: dev-user

B) CA key not found

If ~/.minikube/ca.key doesn't exist, your minikube setup may not expose it (rare).
Workaround: use a **ServiceAccount** based lab instead (tell me if you want that variant).